Supplementary Material

Protocol choice changes the winner more than the ranking: a factorial audit of five negative-set rules and ten peptide bioactivity tasks from a single curated collection

This supplement holds the material the main text refers to but does not need in line: the splitter's procedure and a hand-checkable example, the per-cell balance and leakage audit, the full dataset inventory, the per-level cluster-aware intervals, the pre-specified design simulation, and the inventory of every computation in the deposit. Every number here also appears in the released tables under `results/tables/`. Section numbers cited below refer to the main text.

1 The balance-preserving splitter

The main text introduces this splitter as a post hoc sensitivity instrument and reports that it achieves exact class balance while *raising* residual leakage relative to the size-ordered assignment it replaces. Because that conclusion turns on the selection order, the procedure is given in full below rather than described, and Table S1 works it through on an example small enough to verify by hand.

Table S1: Worked example, small enough to verify by hand: six clusters over 20 sequences (10 positive, 10 negative), $f = 0.20$ and $q = 0.5$, so *need_pos* and *need_neg* both start at 2. The large cluster c_1 is class-pure, which is the situation that produces the artefact on the real corpus. Both resulting partitions are cluster-disjoint, so disjointness is not what distinguishes them.

Cluster	size	positives	negatives	
c_1	8	8	0	large class-pure family
c_2	4	0	4	all-negative family
c_3	2	0	2	
c_4	2	0	2	
c_5	2	1	1	mixed
c_6	2	1	1	mixed

Largest-first takes clusters in descending size until the test fraction is reached. It takes c_1, which alone is $8/20 = 0.40$ of the corpus and entirely positive: achieved test fraction **0.400**, test positive fraction **1.000**.
Quota-driven starts at $(\textit{need_pos}, \textit{need_neg}) = (2, 2)$. Step 1: the quotas tie, so *want_pos* holds; c_1 has $\textit{pos} = 8 > 2$ and does not fit, leaving c_5 and c_6 with $\textit{pos} = 1$, and the jitter picks one, say c_5, giving $(1, 1)$. Step 2: the quotas tie again and only c_6 fits, giving $(0, 0)$, which ends the loop. Test $= c_5 \cup c_6 = 4$ sequences, 2 positive and 2 negative: achieved test fraction **0.200**, test positive fraction **0.500**. Swapping which of c_5, c_6 the jitter takes first exchanges two identical clusters and changes neither number – the sense in which the result does not depend on the jitter.

2 Per-cell balance and leakage

Table S2 is the cell-by-cell version of the summary in the main text: achieved test positive fraction and measured residual pairwise leakage for all 20 cells under both splitters. It is the

```
INPUT   clusters C (from MMseqs2 easy-cluster), labels y,
        test fraction f = 0.20, seed s, target positive fraction q = mean(y)
OUTPUT  test set T, with every cluster entirely inside T or entirely outside

n <- length(y)
for each cluster c:  pos[c] <- #{y=1 in c};  neg[c] <- #{y=0 in c}
                     size[c] <- pos[c] + neg[c]

need_pos <- q * f * n                  # positives still wanted in test
need_neg <- (1 - q) * f * n            # negatives still wanted in test
jitter[c] <- Uniform(0, 1e-6)          # drawn once from seed s; ties only
available <- all clusters;  T <- {}

while (need_pos > 0 or need_neg > 0) and any(available):

    want_pos <- (need_pos >= need_neg)         # serve the further-from-met quota
    counts   <- pos  if want_pos else neg
    quota    <- need_pos if want_pos else need_neg

    fits <- available and counts > 0 and counts <= max(quota, 1)
    pool <- fits  if any(fits)  else  (available and size > 0)   # fallback
    if pool is empty:  break

    best <- argmax over pool of (counts[c] + jitter[c])   # largest that still fits
    T <- T union best                          # clusters are never split
    available[best] <- false
    need_pos <- need_pos - pos[best]
    need_neg <- need_neg - neg[best]

assert every cluster is wholly inside T or wholly outside T
report achieved test fraction and achieved test positive fraction
```

Figure S1: The balance-preserving cluster-disjoint splitter, as implemented (`bra/balanced_split.py`). The only departure from largest-first assignment is the *order* in which clusters are taken: at each step it serves whichever class is further from its quota, and among the clusters small enough to fit that remaining quota it takes the largest. Cluster atomicity is an assertion, not an aspiration. The jitter is bounded by 10^{-6}, so it can only reorder clusters whose counts are exactly equal: it breaks ties seed-dependently without steering selection. When no remaining cluster fits the quota, the fallback admits any non-empty cluster, which is how the procedure terminates on corpora where exact balance is unreachable; it then reports the balance achieved rather than promising one.

evidence for two statements made there – that the quota-driven order hits 0.500 exactly wherever clustering occurs, with no seed-to-seed variation, and that it is not uniformly better, because in several cells it increases leakage.

Table S2: Per-cell audit of both splitters: achieved test positive fraction (mean over five seeds) and measured residual pairwise leakage. The balance-preserving splitter hits 0.500 exactly wherever clustering occurs, with zero seed-to-seed variation. It is not uniformly better: in several cells it *raises* residual leakage (e.g. composition matched at 0.7, 0.043 → 0.100), because the clusters that balance the classes are not the clusters that minimise cross-partition similarity. We report that rather than presenting the correction as free.

		test positive fraction		residual leakage	
Negative rule	thr.	largest-first	balance-pres.	largest-first	balance-pres.
random UniProt	0.9	0.416	**0.500**	0.017	0.016
	0.7	0.500	**0.500**	0.169	0.168
	0.4	0.305	**0.500**	0.393	0.353
	random	0.499	0.499	0.000	0.000
annotation filtered	0.9	0.410	**0.500**	0.018	0.018
	0.7	0.487	**0.500**	0.184	0.184
	0.4	0.288	**0.500**	0.381	0.396
	random	0.499	0.499	0.000	0.000
length matched	0.9	0.574	**0.500**	0.012	0.012
	0.7	0.979	**0.500**	0.046	0.093
	0.4	**0.995**	**0.500**	0.102	0.136
	random	0.499	0.499	0.000	0.000
composition matched	0.9	0.575	**0.500**	0.013	0.012
	0.7	0.978	**0.500**	0.043	0.100
	0.4	0.980	**0.500**	0.110	0.143
	random	0.499	0.499	0.000	0.000
shuffled positives	0.9	0.575	**0.500**	0.011	0.011
	0.7	0.982	**0.500**	0.043	0.090
	0.4	0.991	**0.500**	0.097	0.125
	random	0.499	0.499	0.000	0.000

Achieved test fraction is 0.200–0.201 in every cell under both splitters. Seed-to-seed standard deviation of the positive fraction: ≤ 0.0085 (largest-first), 0.0000 (balance-preserving, clustered cells). Per-cell values with standard deviations are in `tables/armB_balance_leakage_percell_*.csv`.

3 Dataset inventory

Table S3 lists all sixteen released datasets with their sizes as published, their cross-split exact duplicates, and the sequences carrying both labels inside one partition. Ten were used for the ranking analysis; the duplicate audit covers all sixteen because it requires no model fits.

4 Cluster-aware score intervals

Table S4 gives the per-level interval widths behind the summary in the main text, comparing a bootstrap over test sequences with one over whole similarity clusters. Per-pipeline values, including both endpoints for every pipeline, are in `tables/cluster_intervals.csv`.

Table S3: All 16 released datasets: sizes as published, cross-split exact duplicates, and sequences carrying both labels within one partition. The ten used for the ranking analysis are those with ≥ 900 training sequences (stated in the main text's Methods); the duplicate audit covers all 16 because it needs no model fits. **The counts actually scored** are the released sizes minus the test-side duplicates: 52,316 → **52,001** over the ten used, removing 315 sequences.

		as released		duplicates		both labels	in ranking
Task	property	train	test	n	% test	train / test	analysis
AB	antibacterial	13,245	3,311	77	2.3	295 / 31	yes
AMP	antimicrobial	10,266	2,566	0	0.0	7 / 0	yes
AV	antiviral	4,711	1,177	117	9.9	1 / 0	yes
Neuro	neuropeptide	3,881	969	10	1.0	24 / 2	yes
TOX	toxicity	3,092	772	0	0.0	29 / 0	yes
ACE	ACE-inhibitory	1,685	421	26	6.2	33 / 3	yes
AF	antifungal	1,591	395	8	2.0	6 / 0	yes
ACP	anticancer	1,378	344	15	4.4	0 / 0	yes
DPPIV	DPP-IV inhibitory	1,063	265	16	6.0	29 / 3	yes
TTCA	tumour T-cell antigen	948	236	46	**19.5**	0 / 0	yes
AOX	antioxidant	698	174	12	6.9	13 / 0	no
APP	anti-parasitic	482	120	6	5.0	0 / 0	no
QS	quorum sensing	349	87	0	0.0	0 / 0	no
MRSA	anti-MRSA	237	59	3	5.1	0 / 0	no
AMAP	anti-MAP	221	55	0	0.0	0 / 1	no
BBP	blood-brain barrier	191	47	0	0.0	0 / 0	no
ten used			**52,316**	**315**			
all sixteen			55,036	336		11 of 16 affected	

Table S4: Mean 95% interval width on MCC, resampling test sequences against resampling test clusters, with the ratio. The correction tracks how redundant the test partition is: *seq/clu* is the mean number of test sequences per cluster. Arm A's native partitions are already almost non-redundant at this identity, so there the two bootstraps agree.

	level	n_{test}	clusters	seq/clu	mean width (MCC)	
					sequence	**cluster**
Arm B	annotation filtered	2075	312	6.7	0.033	**0.089**
	random UniProt	2076	319	6.5	0.038	**0.076**
	composition matched	2075	620	3.3	0.123	0.163
	length matched	2075	641	3.2	0.135	0.152
	shuffled positives	2075	636	3.3	0.091	0.099
Arm A	ACE-inhibitory	395	388	1.0	0.168	0.166
	antifungal	381	344	1.1	0.190	0.208
	anticancer	325	309	1.1	0.203	0.218
	DPP-IV	249	238	1.0	0.214	0.213
	tumour T-cell antigen	189	181	1.0	0.230	0.244

Mean width ratio (cluster/sequence): Arm B 1.63 on MCC and 1.45 on AUROC; Arm A 1.04 and 1.05. Per-pipeline values in `tables/cluster_intervals.csv`.

5 Design simulation

Figure S2 is the pre-specified design simulation: the agreement ceiling as a function of run noise, and the minimum detectable disagreement as a function of zoo size. It was computed before any benchmark was acquired, and it fixed the minimum detectable disagreement of $\tau_b \leq 0.646$ that the main text quotes when reporting an observation as inconclusive.

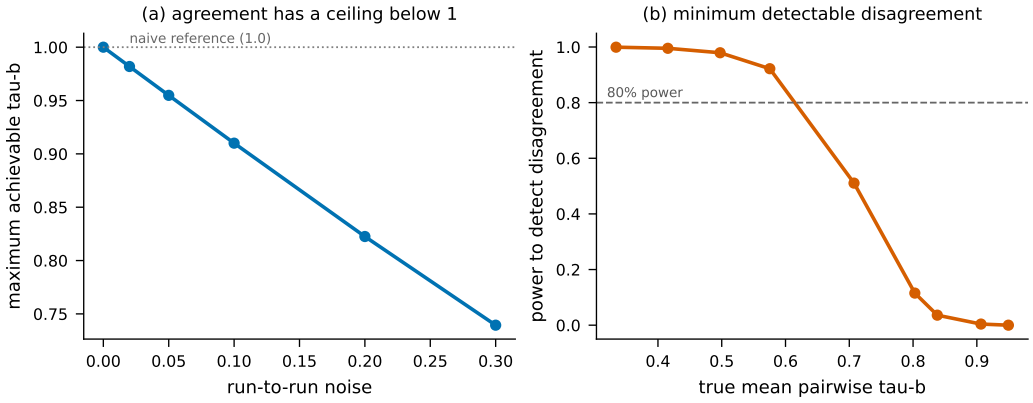


Figure S2: Pre-specified design simulation: the agreement ceiling and the minimum detectable disagreement.

6 Inventory of computations in the deposit

Table S5 accounts for every computation in the archive and reconciles the "4,400 run records" quoted in the abstract with the supplementary analyses added afterwards. Fit counts are recovered from the deposited records rather than from design arithmetic.

Table S5: Every computation in the archive. The 4,400 figure quoted in the abstract is the first block only; the rest are supplementary analyses, deposited alongside it. Fit counts are recovered from the deposited records rather than from design arithmetic.

Analysis	Design	Fits	Script
Main designs – the 4,400 run records			
Arm B, largest-first	100 cells × 16 pipelines	1,600	`run_arm_b.py`
Arm B, balance-preserving	100 cells × 16 pipelines	1,600	`run_arm_b.py`
Arm A	10 tasks × 24 × 5 seeds	1,200	`run_arm_a.py`
subtotal		**4,400**	`runs/all_runs_arm*.parquet`
Supplementary analyses – also deposited			
Equal tuning budget	5 rules × 16 × (6 + 1)	560	`run_equal_tuning.py`
Negative-draw sensitivity	5 rules × 5 draws × 12	300	`run_sensitivity_draw.py`
Cluster-aware intervals	5 rules × 16; 5 tasks × 24	200	`run_cluster_intervals.py`
SVM exact vs Nyström	5 rules × 3 seeds × 2	30	`check_svm_approximation.py`
Splitter stress test	15 cells × 4 splitters	0	`run_splitter_stresstest.py`
Provenance overlap audit	exact set intersection	0	`tables/esm_exposure_*.csv`
total model fits		**5,490**	

The last two fit no models: the stress test performs 60 cluster assignments and 60 leakage searches over cached clusterings, and the audit is an exact set intersection. The cluster-interval analysis additionally performs $2 \times 2{,}000$ bootstrap resamples per fit. `reproduce.sh` runs every stage above in order.